
E-tiGO City Management API

Backend & Frontend Integration Guide

Version 2.0 • October 08, 2026 • Branch: feature/city-management-enhancements

PR #29 • github.com/oluwaeinstein007/eTiGO-api/pull/29

Contents

1. Overview of Changes
 2. Architecture Decisions & Edge Cases
 3. API Endpoints Reference
 4. Create City — Full Walkthrough
 5. Update City — Full Walkthrough
 6. Mapbox Integration (replacing Google Maps)
 7. Frontend Step-by-Step Implementation
 8. Postman Collection & Environment
-

1. Overview of Changes

This document describes the complete city management system for E-tiGO's ride-hailing platform. It covers what the backend handles automatically, what the frontend must send, and how Mapbox replaces Google Maps for all geocoding and directions.

What Changed

Area	Before	After
City fields	name, boundary, timezone, currency_code	+ state (required), region (auto-derived), area_sq_km (auto-calculated)
Vehicle classes	Separate PUT endpoint after city creation	Accepted in create/update payload via vehicle_class_ids
timezone / currency	Required from frontend	Defaults to Africa/Lagos and NGN server-side
State validation	No state field	Validated against 36 Nigerian states + FCT
Region	No region field	Auto-derived from state (e.g. Lagos → South-West)
Map provider	Google Maps (Distance Matrix, Geocoding)	Mapbox (Directions v5, Geocoding v6)
Lookup data	None	GET /lookup/states and GET /lookup/regions
City detection	Point boundary only, no overlap handling	Point + Polygon support, smallest-area overlap resolution

2. Architecture Decisions & Edge Cases

Why region is auto-derived, not sent by frontend

Each Nigerian state belongs to exactly one geopolitical zone. Sending both state and region opens the door to inconsistent data (e.g. state: "Kano", region: "South-West"). The backend derives region from state using a canonical mapping, making mismatches impossible.

Edge Case: Multiple cities in one state

The operational unit is the city, not the state. You can create multiple cities within the same state, each with its own boundary polygon, pricing config, and vehicle classes. For example:

- "Ikeja" in Lagos state — boundary covers Ikeja GRA, Alausa, Maryland
- "Lekki" in Lagos state — boundary covers Lekki Phase 1, Chevron, Ajah
- "VI-Ikoyi" in Lagos state — boundary covers Victoria Island and Ikoyi

Each city gets its own pricing, vehicle classes, and service area. State/region are organizational metadata for filtering and reporting — they do not control pricing or coverage.

Edge Case: Different pricing within same state

Pricing is configured per city via the PricingConfig model (city_id + vehicle_class_id). Two cities in the same state can have completely different base fares, per-km rates, and surge rules. The state field has no effect on pricing.

Edge Case: Service area boundary types

The boundary field supports two GeoJSON types:

- **Point + radius_km** — a circular service area centered on coordinates. Simpler for initial city setup.
- **Polygon** — a custom shape drawn by the admin on the map. The backend auto-calculates area_sq_km from the polygon vertices using the spherical excess formula.

area_sq_km is only computed for Polygon boundaries. For Point boundaries, it remains null because the radius already defines the coverage.

Edge Case: Invalid or unknown state values

The backend validates state against the canonical list of 37 values (36 states + FCT). A request with state: "Unknown" or state: "Lag" returns 422 with the error: "The selected state is invalid." The frontend should populate the state dropdown from the GET /lookup/states endpoint to prevent this.

Edge Case: Overlapping city boundaries

When two or more city/service-area boundaries overlap, a single coordinate can fall inside multiple cities. The /cities/detect endpoint resolves this by collecting all matching candidates and returning the smallest (most specific) city:

- **Point boundaries** — area is calculated as $\pi * \text{radius_km}^2$

- **Polygon boundaries** — area comes from the stored `area_sq_km` (or recalculated if missing)
- All matching candidates are sorted by area ascending; the smallest wins

Example: If "Lagos Metro" (radius 50km) and "Ikeja" (radius 5km) both contain the rider's coordinates, `/cities/detect` returns "Ikeja" because it has the smaller area. This lets you define a large metro-wide fallback zone and precise inner zones without conflict.

Edge Case: City detection with both boundary types

The `/cities/detect` endpoint fully supports both Point + radius and Polygon boundaries in the same deployment. Detection works as follows:

- **Step 1** — Reverse-geocode the coordinates via Mapbox to get a city name
- **Step 2** — Match the name against active cities in the database
- **Step 3** — If no name match, fall back to boundary matching:
 - Point: haversine distance $\leq$ `radius_km`
 - Polygon: ray-casting point-in-polygon algorithm
- **Step 4** — If multiple boundaries match, return the smallest (overlap resolution)

Both boundary types can coexist. A city with a Polygon boundary and a city with a Point boundary can overlap; the detection algorithm handles them uniformly. The admin is free to start with a simple Point + radius for quick setup and later upgrade to a precise Polygon.

Edge Case: Slug collisions

If two cities produce the same slug (e.g. "Lagos" and "LAGOS"), the backend appends a numeric suffix: `lagos`, `lagos-1`. This is handled automatically.

3. API Endpoints Reference

Public Endpoints (no auth)

Method	Endpoint	Description
GET	/api/v1/lookup/states	List all 37 Nigerian states with codes and regions
GET	/api/v1/lookup/regions	List 6 geopolitical zones with member states
GET	/api/v1/cities	List active cities (public, for passenger/driver apps)
GET	/api/v1/cities/detect	Detect city from lat/lng coordinates
GET	/api/v1/cities/{city}/vehicle-classes	List vehicle classes for a city

Admin Endpoints (requires admin token)

Method	Endpoint	Description
GET	/api/v1/admin/cities	List cities with filters: search, is_active, state, region
POST	/api/v1/admin/cities	Create a new city
GET	/api/v1/admin/cities/{city}	Get city details with vehicle classes
PUT	/api/v1/admin/cities/{city}	Update a city
PATCH	/api/v1/admin/cities/{city}/status	Toggle city active/inactive
PUT	/api/v1/admin/cities/{city}/vehicle-classes	Update vehicle classes (legacy, still works)
GET	/api/v1/admin/vehicle-classes	List all vehicle classes

4. Create City — Full Walkthrough

Request

POST `/api/v1/admin/cities`

Content-Type: application/json | Authorization: Bearer {admin_token}

Payload (what the frontend sends)

```
{
  "name": "Ikeja",
  "state": "Lagos",
  "boundary": {
    "type": "Polygon",
    "coordinates": [[
      [3.3350, 6.6200],
      [3.3700, 6.6200],
      [3.3700, 6.5800],
      [3.3350, 6.5800],
      [3.3350, 6.6200]
    ]]
  },
  "vehicle_class_ids": [
    "a1b2c3d4-...",
    "e5f6g7h8-..."
  ]
}
```

Field Reference

Field	Type	Required	Notes
name	string	Yes	Max 255 chars, must be unique
state	string	Yes	Must be a valid Nigerian state (from <code>/lookup/states</code>)
boundary	object	No	GeoJSON: Point or Polygon (see below)
vehicle_class_ids	array	No	UUIDs from <code>GET /admin/vehicle-classes</code>
timezone	string	No	Defaults to Africa/Lagos
currency_code	string	No	Defaults to NGN (3 chars)

What the backend does automatically

- **region** — Derived from state (Lagos → South-West, Kano → North-West)
- **slug** — Generated from name ("Ikeja" → "ikeja"), auto-suffixed if duplicate
- **area_sq_km** — Calculated from Polygon boundary coordinates
- **is_active** — Defaults to true
- **timezone** — Defaults to Africa/Lagos if not sent

- **currency_code** — Defaults to NGN if not sent

Response (201 Created)

```
{
  "message": "City created successfully.",
  "city": {
    "id": "uuid",
    "name": "Ikeja",
    "slug": "ikeja",
    "state": "Lagos",
    "region": "South-West",
    "boundary": { ... },
    "area_sq_km": "12.45",
    "timezone": "Africa/Lagos",
    "currency_code": "NGN",
    "is_active": true,
    "vehicle_classes": [ ... ],
    "created_at": "...",
    "updated_at": "..."
  }
}
```

5. Update City — Full Walkthrough

PUT /api/v1/admin/cities/{city_id}

All fields are optional. Only send the fields you want to change. If state is updated, region is automatically re-derived.

```
{
  "name": "Ikeja GRA",
  "state": "Lagos",
  "boundary": {
    "type": "Polygon",
    "coordinates": [[ ... ]]
  },
  "vehicle_class_ids": ["uuid-1", "uuid-2", "uuid-3"]
}
```

Sending vehicle_class_ids replaces the current set entirely (sync). Omitting the field keeps the existing vehicle classes unchanged.

Toggle Status

PATCH /api/v1/admin/cities/{city_id}/status

No request body needed. Toggles is_active between true and false. Deactivated cities are hidden from the passenger/driver app but remain in the admin panel.

6. Mapbox Integration

What changed

Feature	Before (Google Maps)	After (Mapbox)
Directions / Distance	Distance Matrix API	Directions v5 API
Forward Geocoding	Geocoding API	Geocoding v6 (forward)
Reverse Geocoding	Geocoding API	Geocoding v6 (reverse)
City Detection	Reverse geocode → extract city name	Same logic, Mapbox context.place.name
Env variable	GOOGLE_MAPS_API_KEY	MAPBOX_ACCESS_TOKEN
Fallback	HaversineMapsGateway	Same — still HaversineMapsGateway

Backend architecture

The backend uses a MapsGateway interface (contract pattern). The AppServiceProvider binds MapboxGateway when MAPBOX_ACCESS_TOKEN is set, otherwise falls back to HaversineMapsGateway (straight-line distance estimation). Frontend and backend use the same Mapbox access token.

Environment setup

```
# .env
MAPBOX_ACCESS_TOKEN=pk.eyJ1Ijoi...
```

Frontend Mapbox setup

For the admin panel map widget (drawing service area boundaries), use Mapbox GL JS:

```
# Install
npm install mapbox-gl @mapbox/mapbox-gl-draw

# In your map component:
import mapboxgl from "mapbox-gl";
import MapboxDraw from "@mapbox/mapbox-gl-draw";

mapboxgl.accessToken = process.env.MAPBOX_ACCESS_TOKEN;

const map = new mapboxgl.Map({
  container: "map",
  style: "mapbox://styles/mapbox/streets-v12",
  center: [3.3792, 6.5244], // Lagos
  zoom: 11,
});

const draw = new MapboxDraw({
  displayControlsDefault: false,
  controls: { polygon: true, trash: true },
});
```

```
map.addControl(draw);
```

Extracting the polygon for the API

```
map.on("draw.create", (e) => {  
  const polygon = e.features[0];  
  const boundary = {  
    type: "Polygon",  
    coordinates: polygon.geometry.coordinates,  
  };  
  // Send this as the boundary field in the API payload  
});
```

7. Frontend Step-by-Step Implementation

This section walks through implementing the "Add City" form as shown in the Figma design.

Step 1: Fetch dropdown data on form mount

When the "Add City" modal opens, make three parallel API calls to populate the dropdowns:

```
const [states, regions, vehicleClasses] = await Promise.all([
  fetch("/api/v1/lookup/states").then(r => r.json()),
  fetch("/api/v1/lookup/regions").then(r => r.json()),
  fetch("/api/v1/admin/vehicle-classes", {
    headers: { Authorization: `Bearer ${token}` }
  }).then(r => r.json()),
]);
```

Step 2: Build the form fields

Design Field	Input Type	Data Source	API Field
City name	Text input	User types	name
State	Dropdown	GET /lookup/states	state
Region	Auto-filled (read-only)	Derived from selected state	Not sent
Service area size	Auto-filled (read-only)	Calculated by backend	Not sent
Status	Not in create form	Defaults to active	Not sent
Vehicle classes	Toggle chips	GET /admin/vehicle-classes	vehicle_class_ids
Service area boundary	Mapbox GL Draw	User draws polygon	boundary

Step 3: Auto-fill region from state selection

When the user selects a state, look up its region from the states response and display it as read-only:

```
function onStateChange(selectedStateName) {
  const state = states.states.find(
    s => s.name === selectedStateName
  );
  setRegion(state?.region ?? "");
}
```

Step 4: Initialize the Mapbox map with draw controls

Render a Mapbox GL map inside the modal. Add the polygon draw tool. Center the map on Nigeria (lat: 9.08, lng: 7.49, zoom: 6) and let the admin zoom into the city.

Step 5: Capture the drawn polygon

Listen for `draw.create` and `draw.update` events. Extract the GeoJSON coordinates:

```
function onDrawUpdate(e) {
  const feature = e.features[0];
  if (feature.geometry.type === "Polygon") {
    setBoundary({
      type: "Polygon",
      coordinates: feature.geometry.coordinates,
    });
  }
}
```

Step 6: Submit the form

```
async function handleSubmit() {
  const payload = {
    name: cityName,
    state: selectedState,
    boundary: boundary, // from map
    vehicle_class_ids: selectedVehicleClassIds,
  };

  const res = await fetch("/api/v1/admin/cities", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify(payload),
  });

  const data = await res.json();

  if (res.ok) {
    // Success: data.city has full city with region,
    // area_sq_km, slug, vehicle_classes, etc.
    showToast(data.message);
    closeModal();
    refreshCityList();
  } else {
    // Validation error: data.errors has field errors
    showErrors(data.errors);
  }
}
```

Step 7: Display the response

After successful creation, the response includes all computed fields. Update the city list and show `area_sq_km` and `region` in the city details view. These are read-only display fields.

8. Postman Collection & Environment

The Postman collection and both environments (Local, Staging) have been synced. The collection includes all endpoints listed above with example payloads.

Updated requests

- **Create City** — payload now uses state (no region), includes vehicle_class_ids
- **Update City** — payload includes state and vehicle_class_ids
- **List Nigerian States** — new GET /api/v1/lookup/states
- **List Geopolitical Regions** — new GET /api/v1/lookup/regions

Environment variables

Variable	Local	Staging
base_url	http://localhost:8000/api/v1	https://staging-api.etigo.com/api/v1
admin_token	(set after login)	(set after login)
city_id	(set after creating a city)	(set after creating a city)

Removed environment variable

GOOGLE_MAPS_API_KEY has been removed from .env.example. Replace it with MAPBOX_ACCESS_TOKEN in all environments.

For questions, refer to the Postman collection or reach out to the backend team.